Home Credit Default Risk

Technical Report — End-to-End Machine Learning Pipeline for Loan Default Prediction

Valvitor Santos

April 19, 2026

Abstract. This report describes the development of a complete machine learning pipeline for the *Home Credit Default Risk* Kaggle competition. The problem consists of predicting the probability of loan default for clients who may have limited or no credit history. The solution employs feature engineering across seven relational tables, a LightGBM model with Optuna hyperparameter optimization, and 5-fold stratified cross-validation. Experiments are tracked with MLflow. The evaluation metric is **AUC-ROC**.

Contents

1 Introduction and Problem Context **2**
1.1 Motivation . . . 2
1.2 Problem Formulation . . . 2
1.3 Main Challenges . . . 2

2 Dataset **2**
2.1 Relational Structure . . . 2
2.2 Target Variable . . . 3

3 Exploratory Data Analysis **3**
3.1 Anomaly in `DAYS_EMPLOYED` . . . 3
3.2 EXT_SOURCE Features . . . 3
3.3 Real Estate Block . . . 4
3.4 Class Imbalance . . . 4

4 Feature Engineering **4**
4.1 Overview . . . 4
4.2 Main Table Features . . . 4
4.2.1 Financial Ratios . . . 4
4.2.2 Time-Based Features . . . 5
4.2.3 EXT_SOURCE Combinations . . . 5
4.3 Auxiliary Table Aggregation . . . 5
4.3.1 Bureau and Bureau Balance . . . 5
4.3.2 Installments Payments . . . 6
4.3.3 Credit Card Balance . . . 6
4.3.4 Previous Applications . . . 6

4.4 Final Processing 6

5 Modeling **6**
5.1 Algorithm Selection 6
5.2 Hyperparameter Optimization with Optuna 7
5.3 Final Training with 5-Fold CV 7
5.4 OOF AUC vs. Mean Fold AUC 8
5.5 Experiment Tracking with MLflow 8

6 Engineering Best Practices **8**
6.1 Code Structure 8
6.2 Memory Management 8
6.3 Data Leakage Prevention 9

7 Conclusion and Future Work **9**
7.1 Conclusion 9
7.2 Future Work 9

A Dependencies **10**

1 Introduction and Problem Context

1.1 Motivation

Financial institutions face the challenge of assessing the repayment capacity of clients who have little or no formal credit history. The absence of historical data in conventional credit bureaus leads to the financial exclusion of populations that are, in fact, capable of honoring their commitments. Home Credit, an organization specialized in responsible lending, seeks to use alternative data sources to make this assessment fairer and more accurate.

1.2 Problem Formulation

The objective is to build a **binary classification** model that, given an applicant's profile, estimates the probability of default:

$$\hat{p}_i = P(\text{TARGET}_i = 1 \mid \mathbf{x}_i) \tag{1}$$

where $\text{TARGET}_i = 1$ indicates that client i did not repay the loan, and $\mathbf{x}_i$ is the client's feature vector.

The official evaluation metric is the **AUC-ROC** (Area Under the ROC Curve), which measures the model's ability to separate defaulters from non-defaulters regardless of the classification threshold:

$$\text{AUC} = \int_0^1 \text{TPR}\big(\text{FPR}^{-1}(t)\big)\, dt \tag{2}$$

1.3 Main Challenges

- **Class imbalance:** approximately 8% of contracts result in default, creating strong bias toward the majority class.
- **Data distributed across multiple tables:** relevant information is fragmented across 7 relational tables that must be aggregated to the applicant level.
- **High proportion of missing values:** some features have up to 60% null values, requiring careful treatment.
- **Critical feature engineering:** model performance depends heavily on the quality of derived features, especially from auxiliary tables.

2 Dataset

2.1 Relational Structure

The data is provided in 7 CSV files with the following hierarchy:

Table	Key	Rows	Description
application_train/test	SK_ID_CURR	307,511 / 48,744	Current loan application
bureau	SK_ID_CURR	1,716,428	Credit at other banks
bureau_balance	SK_ID_BUREAU	27,299,925	Monthly bureau history
previous_application	SK_ID_CURR	1,670,214	Prior HC applications
POS_CASH_balance	SK_ID_CURR	10,001,358	POS and cash loan balances
installments_payments	SK_ID_CURR	13,605,401	Repayment history
credit_card_balance	SK_ID_CURR	3,840,312	Credit card balances

Table 1: Dataset structure

The main table is the model anchor; all others are aggregated to the `SK_ID_CURR` level and joined via `left join`.

2.2 Target Variable

- `TARGET` = 0: client repaid normally (91.93%)
- `TARGET` = 1: client defaulted (8.07%)

3 Exploratory Data Analysis

The EDA was conducted in notebook `01_EDA.ipynb` with the goal of extracting actionable insights — not merely descriptive statistics, but pre-processing and feature engineering decisions directly informed by the data.

3.1 Anomaly in `DAYS_EMPLOYED`

The variable `DAYS_EMPLOYED` represents how many days a client has been employed (negative value). However, 18,148 clients show the value **365,243** — clearly a placeholder for "not employed", equivalent to $\approx$ 1,000 years of employment.

Decision: replace 365,243 with `NaN` and create a binary feature `FLAG_EMPLOYED_ANOMALY` to preserve the information that data was missing.

Listing 1: Handling the DAYS_EMPLOYED anomaly

```
df['FLAG_EMPLOYED_ANOMALY'] = (df['DAYS_EMPLOYED'] == 365243).astype(np.int8)
df['DAYS_EMPLOYED'] = df['DAYS_EMPLOYED'].replace(365243, np.nan)
```

3.2 EXT_SOURCE Features

The variables `EXT_SOURCE_1`, `EXT_SOURCE_2`, and `EXT_SOURCE_3` represent external credit scores from third-party bureaus. EDA revealed these are the **most correlated features with default** in the entire dataset, making them priority candidates for non-linear combinations.

`EXT_SOURCE_1` has 56% missing values, requiring an additional flag: `FLAG_EXT_SOURCE_1_MISSING`.

3.3 Real Estate Block

Approximately 20 variables related to the client's property (`APARTMENTS_AVG`, `BASEMENTAREA_AVG`, etc.) have more than 60% null values and low correlation with the target.

Decision: drop all columns with more than 60% missing values, except `EXT_SOURCE`.

3.4 Class Imbalance

With 8.07% default rate, unadjusted models tend to classify almost all clients as non-defaulters. **Decision:** use LightGBM's `scale_pos_weight` parameter:

$$\texttt{scale_pos_weight} = \frac{N_{\text{negatives}}}{N_{\text{positives}}} = \frac{283{,}301}{24{,}210} \approx 11.70 \tag{3}$$

4 Feature Engineering

4.1 Overview

Feature engineering was executed in notebook `02_feature_engineering.ipynb`, transforming 7 raw tables into a single processed dataset with over 1,000 features. The process follows four steps:

1. Manual features on the main application table
2. Bureau and bureau_balance aggregation
3. Auxiliary table aggregation (previous, POS, installments, credit card)
4. Final merge, One-Hot Encoding and alignment

4.2 Main Table Features

4.2.1 Financial Ratios

Ratios capturing the relationship between credit, income, and installments:

Feature	Formula
`CREDIT_INCOME_RATIO`	$\texttt{AMT_CREDIT}/(\texttt{AMT_INCOME_TOTAL} + 1)$
`ANNUITY_INCOME_RATIO`	$\texttt{AMT_ANNUITY}/(\texttt{AMT_INCOME_TOTAL} + 1)$
`ANNUITY_CREDIT_RATIO`	$\texttt{AMT_ANNUITY}/(\texttt{AMT_CREDIT} + 1)$
`INCOME_PER_PERSON`	$\texttt{AMT_INCOME_TOTAL}/(\texttt{CNT_FAM_MEMBERS} + 1)$
`GOODS_CREDIT_RATIO`	$\texttt{AMT_GOODS_PRICE}/(\texttt{AMT_CREDIT} + 1)$

Table 2: Financial features created from the main table

Adding 1 to the denominator avoids division by zero without distorting values for large denominators.

4.2.2 Time-Based Features

- `DAYS_BIRTH_YEARS`: client age in years (more interpretable than negative days)
- `DAYS_EMPLOYED_YEARS`: employment duration in years
- `EMPLOYED_TO_BIRTH_RATIO`: fraction of life spent working
- `EMPLOYED_TO_AGE_PCT`: employment duration / age ratio

4.2.3 EXT_SOURCE Combinations

Given the high predictive power of external scores, multiple combinations were created:

$$\texttt{EXT_SOURCE_MEAN} = \tfrac{1}{3}(E_1 + E_2 + E_3) \tag{4}$$

$$\texttt{EXT_SOURCE_STD} = \mathrm{std}(E_1, E_2, E_3) \tag{5}$$

$$\texttt{EXT_SOURCE_PROD} = E_1 \cdot E_2 \cdot E_3 \tag{6}$$

$$\texttt{EXT12} = E_1 \cdot E_2, \quad \texttt{EXT13} = E_1 \cdot E_3, \quad \texttt{EXT23} = E_2 \cdot E_3 \tag{7}$$

The product captures non-linear interactions: a low score from any single source collapses the product, signaling concentrated risk.

4.3 Auxiliary Table Aggregation

For each auxiliary table, the process is:

1. Create domain-specific manual features for the table
2. Apply statistical aggregations for numeric columns: `count`, `mean`, `max`, `min`, `sum`
3. Apply One-Hot Encoding + aggregation for categorical columns: `sum`, `mean`
4. Left-join to the `SK_ID_CURR` level

This process is implemented in reusable functions `agg_numeric()` and `agg_categorical()` in `src/features.py`.

4.3.1 Bureau and Bureau Balance

`bureau_balance` provides monthly history for each bureau credit entry. Status columns were binarized:

Listing 2: Feature engineering on bureau_balance

```
# C = closed, X = no loan, 1-5 = months overdue (increasing severity)
bureau_balance['STATUS_GOOD'] = bureau_balance['STATUS'].isin(['C','X']).astype(np.int8)
bureau_balance['STATUS_LATE'] = bureau_balance['STATUS'].isin(['1','2','3','4','5']).astype(np.int8)
```

4.3.2 Installments Payments

- `PAYMENT_DIFF`: amount paid − amount due (positive = paid more than the minimum)
- `PAYMENT_RATIO`: ratio of amount paid to amount due
- `DAYS_PAST_DUE`: days overdue (clipped to 0 for early payments)
- `PAID_ON_TIME`: binary on-time payment flag

4.3.3 Credit Card Balance

$$\texttt{UTILIZATION} = \frac{\texttt{AMT_BALANCE}}{\texttt{AMT_CREDIT_LIMIT_ACTUAL} + 1} \tag{8}$$

High credit utilization is a strong indicator of financial stress.

4.3.4 Previous Applications

- `APP_CREDIT_RATIO`: ratio of amount applied for to amount approved — captures whether clients typically request more than they receive
- `DAYS_DECISION_YEARS`: time since last credit decision, in years

4.4 Final Processing

After merging all tables:

1. Drop columns with **more than 80% nulls** post-merge (auxiliary tables introduce many nulls for clients without history in a given product)
2. **One-Hot Encoding** for categorical variables with fewer than 10 unique categories
3. **Alignment** between train and test: columns missing in test are filled with 0
4. **Column name sanitization**: special characters replaced by _ (required for LightGBM compatibility)
5. Save to **Parquet format** for efficient I/O

The final result is a dataset with **1,042 features** and 307,511 training rows.

5 Modeling

5.1 Algorithm Selection

LightGBM (Light Gradient Boosting Machine) was chosen for the following reasons:

- **Efficiency with high dimensionality:** the GOSS and EFB algorithms allow fast training with over 1,000 features
- **Native missing value handling:** learns the best split direction for missing data without imputation

- **Performance on tabular data:** gradient boosting trees historically outperform neural networks on tabular data with domain-engineered features
- **Imbalance support:** `scale_pos_weight` adjusts the minority class weight in gradient computation

5.2 Hyperparameter Optimization with Optuna

Optuna implements the **TPE** (Tree-structured Parzen Estimator), a Bayesian optimization method that builds a probabilistic model of the hyperparameter space and suggests configurations with the highest probability of improvement over the current best.

The tuning process used:

- 20 trials, each evaluated with 3-fold CV
- Objective metric: mean AUC-ROC across the 3 folds
- Early stopping of 30 rounds per fold to speed up the search

Hyperparameter	Search Space	Type
`learning_rate`	[0.02, 0.15]	log-uniform
`num_leaves`	[20, 200]	integer
`max_depth`	[4, 10]	integer
`min_child_samples`	[20, 300]	integer
`feature_fraction`	[0.4, 0.9]	uniform
`bagging_fraction`	[0.4, 0.9]	uniform
`bagging_freq`	[1, 7]	integer
`reg_alpha`	[10^{-4}, 5.0]	log-uniform
`reg_lambda`	[10^{-4}, 5.0]	log-uniform

Table 3: Optuna search space

5.3 Final Training with 5-Fold CV

After identifying the best hyperparameters, the final model is trained with **StratifiedKFold** of 5 folds, which preserves the default rate proportion in each fold:

Listing 3: Stratified cross-validation training

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
oof_preds  = np.zeros(len(X))
test_preds = np.zeros(len(X_test))

for fold, (tr_idx, val_idx) in enumerate(skf.split(X, y)):
    model = lgb.LGBMClassifier(**BEST_PARAMS)
    model.fit(X.iloc[tr_idx], y.iloc[tr_idx],
              eval_set=[(X.iloc[val_idx], y.iloc[val_idx])],
              callbacks=[lgb.early_stopping(100)])
    oof_preds[val_idx]  = model.predict_proba(X.iloc[val_idx])
   [:, 1]
```

```
    test_preds          += model.predict_proba(X_test)[:, 1] / 5
```

Test predictions are the **average** across the 5 models, reducing variance.

5.4 OOF AUC vs. Mean Fold AUC

The **OOF AUC** (Out-of-Fold) is computed by concatenating all out-of-fold predictions and calculating AUC over the entire training set at once:

$$\mathrm{AUC}_{\mathrm{OOF}} = \mathrm{AUC}(\mathbf{y}_{\mathrm{train}},\ \hat{\mathbf{p}}_{\mathrm{OOF}}) \tag{9}$$

This differs from the mean of per-fold AUCs:

$$\overline{\mathrm{AUC}}_{\mathrm{folds}} = \frac{1}{K}\sum_{k=1}^{K} \mathrm{AUC}\Big(\mathbf{y}_{\mathrm{val}}^{(k)},\ \hat{\mathbf{p}}_{\mathrm{val}}^{(k)}\Big) \tag{10}$$

The OOF AUC is more reliable because it evaluates the **global ranking** of all predictions together. Individual folds may show higher AUC due to statistically more separable data — the OOF metric corrects for this bias.

5.5 Experiment Tracking with MLflow

All experiments are logged in MLflow:

- Best Optuna trial hyperparameters
- Final OOF AUC and per-fold AUC
- Number of features used

This enables reproducible comparison of future runs with new features or different models.

6 Engineering Best Practices

6.1 Code Structure

The separation between notebooks and reusable code (`src/features.py`) avoids duplication. The functions `agg_numeric()` and `agg_categorical()` are generic — they accept any DataFrame, primary key, and name prefix, making them applicable to all auxiliary tables without modification.

6.2 Memory Management

With datasets of tens of millions of rows, memory usage is critical:

- **Type downcasting:** integers and floats are converted to the smallest type that fits their range (`int8`, `int16`, `float32`), typically achieving 40–60% memory reduction

- **Explicit garbage collection:** after each auxiliary table is processed, the object is deleted and `gc.collect()` is called
- **Parquet:** columnar, compressed intermediate storage instead of CSV

6.3 Data Leakage Prevention

Train-test column alignment is performed via `pd.DataFrame.align(join='left')` to ensure the test set never contains features derived from target information. When Target Encoding is applied, K-Fold is used to prevent the target from leaking into the validation fold.

7 Conclusion and Future Work

7.1 Conclusion

This project implements a robust end-to-end pipeline for loan default prediction. The main technical highlights are:

1. Decision-driven EDA, not merely descriptive
2. Feature engineering grounded in financial domain knowledge
3. Stratified cross-validation to handle class imbalance
4. Bayesian hyperparameter optimization with Optuna
5. Full experiment tracking with MLflow

7.2 Future Work

- **Stacking/Blending:** combine LightGBM with XGBoost and CatBoost to further reduce variance
- **Target Encoding:** apply to high-cardinality variables such as `ORGANIZATION_TYPE`
- **SHAP values:** feature selection based on permutation importance to remove noisy features
- **Pseudo-labeling:** use high-confidence test predictions to augment the training set

A Dependencies

Library	Version
pandas	2.2.2
numpy	1.26.4
scikit-learn	1.4.2
lightgbm	4.x
optuna	3.6.1
mlflow	2.11.3
pyarrow	15.0.2
matplotlib	3.8.4
seaborn	0.13.2

Table 4: Main project dependencies